Наследование и runtime polymorphism

Перед выполнением изучить главы 05 «Наследование и виртуальные функции» и 06 «Статический и динамический полиморфизм»

Задание

Разработать программу для расчета парковочных сессий платной парковки

Зоны: `A` - открытая парковка, `B` - крытая парковка, `C` - грузовая зона. Для типов транспорта действуют правила:

Тип	Код в файле	Места	Зоны	Рублей за час
автомобиль	`CAR`	1	A, B	120
мотоцикл	`MOTORCYCLE`	1	A, B	60
микроавтобус	`MINIBUS`	2	B	180
грузовик	`TRUCK`	3	C	250 + 40 за тонну груза

Длительность считать в минутах. Неполный час оплачивается как полный, минимальная оплата - один час. Масса груза задается в тоннах и может быть дробной

Журнал `parking_log.txt` содержит события одного дня. Формат строк:

```
ENTER время номерной_знак тип зона
ENTER время номерной_знак TRUCK зона масса_груза
EXIT время номерной_знак
```

Например:

```
ENTER 08:10 A123BC CAR A
ENTER 09:00 M777MM MOTORCYCLE B
EXIT 09:40 M777MM
EXIT 10:05 A123BC
ENTER 11:20 B456TY MINIBUS B
EXIT 14:05 B456TY
ENTER 07:30 T900TT TRUCK C 8
EXIT 12:10 T900TT
ENTER 13:00 C111CC CAR C
EXIT 14:00 C111CC
ENTER 15:15 T222AA TRUCK B 5
EXIT 16:45 T222AA
```

Каждый `ENTER` начинает сессию, `EXIT` завершает активную сессию по номерному знаку. Формат строк считать корректным. Для каждого въезда есть выезд с временем не раньше въезда. После завершения сессии номер может встретиться снова. Общая хронологическая сортировка разных сессий не требуется

Создать общий абстрактный интерфейс транспорта и четыре производных класса. Проверка зоны, количество мест и стоимость должны определяться через этот интерфейс

Парковочная сессия содержит транспорт и данные посещения. Одна коллекция должна хранить сессии для всех типов транспорта. Владение транспортом организовать через `std::unique_ptr` на базовый тип, вызов деструктора конкретного транспорта - через виртуальный деструктор базового класса

Для каждой сессии вывести номер, тип, зону, длительность в минутах, оплачиваемые часы, количество мест и стоимость. Для недопустимой зоны вывести причину ошибки вместо стоимости

В конце вывести общую выручку, сумму мест по допустимым сессиям и количество недопустимых сессий. Сумма мест учитывает каждое допустимое посещение, а не максимальную одновременную загрузку. Программа читает файл и формирует отчет без интерактивного меню

Требования

- Названия сущностей программы должны быть осмысленными и записываться на английском языке без транслитерации
- Использовать абстрактный базовый класс, public inheritance, `override` и открытый виртуальный деструктор
- Сессия должна содержать транспорт через composition и владеть им через `std::unique_ptr<Base>`. Хранение производных объектов как значений базы в основной модели запрещено
- Показать обработку через ссылку или указатель на базовый тип
- Минимальный набор операций: чтение события, проверка зоны, расчет длительности и расчет стоимости через общий интерфейс
- Разбить программу как минимум на один `.hpp` и два `.cpp` файла. Для headers использовать `#pragma once`
- Обработать ошибку открытия файла. Программа должна компилироваться и выполняться без ошибок
- Студент должен понимать код, обосновывать решение и отвечать на вопросы во время защиты